Instituto Tecnológico de Costa Rica
Campus Tecnológico Local San Carlos
Escuela de Ingeniería en Computación

IC-6600 Principios de Sistemas Operativos · Grupo 51
II Proyecto Programado · Alternativa A

# NeonArch

## A custom Linux distribution on Arch Linux, built, instrumented and measured

Final report

| | |
|---|---|
| Hernán Alonso Gutiérrez Quesada | 2023-800-185 |
| Royner Baltodano Sánchez | *carné* |

Professor: Jorge Alberto Alfaro Velasco

Repository: `https://github.com/Hernan1787/neonarch`

November 2026

# Summary

NeonArch is not a downloadable image: it is a reproducible layer of configuration, scripts, systemd units, pacman hooks and own commands applied by an idempotent bootstrap (`install.sh`) on top of an Arch Linux system that each member installed by hand (UEFI, btrfs with subvolumes, systemd-boot). Seven operating-system topics were implemented and *measured*; this report integrates the technical decisions, the measurements, the limitations we found and the lessons learned. Several results contradicted our first expectations (an "optimisation" that made the boot slower, a benchmark that was measuring RAM instead of the disk, a scheduler behaviour that changed in recent kernels); in every case we kept the data and explained it rather than discarding it.

| Environment | Value |
|---|---|
| Host | AMD Ryzen 5 3600X (6C/12T), 16 GB RAM, NVMe SSD, Windows 11, VirtualBox 7.2 |
| VM | 2 vCPU, 4 GiB RAM, 40 GiB VDI, UEFI (OVMF), VMSVGA without 3D, NAT |
| OS | Arch Linux, kernel 7.2.15-arch1-1, systemd 262, pacman 7.1.0, btrfs root |
| Desktop | i3 window manager on X11, xterm, rofi, i3status, dunst ("Neon" theme) |

# Contents

**1 Layer 1 — Base installation and reproducibility** **2**
- 1.1 Manual installation decisions . . . 2
- 1.2 Idempotent bootstrap . . . 2
- 1.3 Desktop: i3, justified with data . . . 2

**2 Layer 2 — Operating-system topics** **3**
- 2.1 Topic 1 — Processes and scheduling . . . 3
- 2.2 Topic 2 — Memory . . . 4
- 2.3 Topic 3 — Filesystem (btrfs) . . . 4
- 2.4 Topic 4 — Service management (systemd) . . . 5
- 2.5 Topic 6 — Package management (pacman) . . . 6
- 2.6 Topic 7 — Boot process . . . 6
- 2.7 Topic 9 — Protection and security . . . 7

**3 Layer 3 — Own commands and shortcuts** **7**

**4 Limitations** **8**

**5 Lessons learned** **8**

**6 Conclusions** **9**

# 1 Layer 1 — Base installation and reproducibility

## 1.1 Manual installation decisions

Each member installed Arch by hand following `docs/instalacion.md`, which explains *why* each command is run (`archinstall` was not used). Table 1 summarises the decisions that later topics depend on.

| Decision | Reason |
|---|---|
| UEFI + GPT, 1 GiB FAT32 ESP | Required by the assignment; the firmware reads EFI executables only from a FAT ESP. 1 GiB holds several kernels/initramfs (systemd-boot loads them from the ESP). |
| btrfs root with subvolumes `@ @home @snapshots @var_log @pkg @swap` | Topic 3. A snapshot of `@` excludes user files, logs, package cache and swap, so a rollback restores the system without losing data or the logs that explain what broke. |
| Mount options `noatime`, `compress=zstd:1`, `space_cache=v2`; `subvol=` only, no `subvolid=` | Avoid a CoW metadata write on every read; transparent compression; a rolled-back `@` has a new ID, so pinning the ID would break the boot. |
| systemd-boot | Plain-text entries; easy to add kernel parameters and rescue entries (Topic 7). |
| ESP mounted `fmask=0077,dmask=0077` | Only root can read the bootloader files (Topic 9). |
| Normal user in `wheel` + sudo | Least privilege; every privileged action is logged. |

**Table 1:** Installation decisions (details and the reason for every command: `docs/instalacion.md`).

**Problems found.** The installation log records 14 real problems with cause and fix: virtualisation (AMD-V/SVM) disabled in the BIOS — it was disabled again later by a firmware reset —, typos that created no subvolume or a wrong kernel parameter (`rm` instead of `rw`), a `sed` expression without the file separator that silently generated no locale, a `loader.conf` written to the wrong path, the bootloader not registering its NVRAM entry from inside `arch-chroot` (fixed after the first boot with `bootctl install`), and two problems of the desktop described below. Current Arch no longer builds the *fallback* initramfs, so the guide was corrected to create a single boot entry.

## 1.2 Idempotent bootstrap

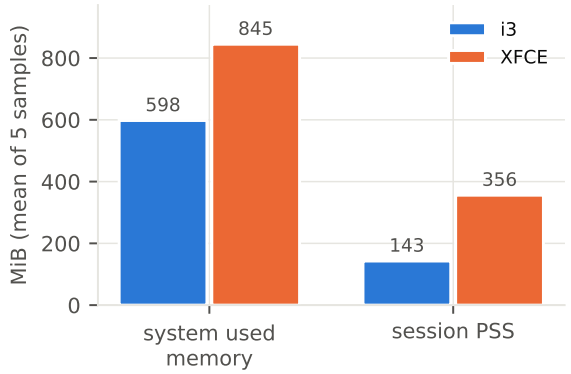
`install.sh` turns a clean manual install into NeonArch in six stages; each stage first compares the current state with the desired one and acts only if they differ (Table 2). Every run writes a log to `logs/`. On `neonarch-hernan` the first full run applied 21 changes and the next run reported *"system already up to date: 0 changes (idempotent run)"* (`logs/install-neonarch-hernan-20261110-231929.log`).

| Stage | Action | Idempotency check |
|---|---|---|
| packages | install `pkglist.txt` (+ VM guest tools by `systemd-detect-virt`) | `pacman -T` lists only unsatisfied packages |
| dotfiles, commands | symlink `dotfiles/*` into `$HOME`, `bin/*` into `~/.local/bin` | skip links that already point to the repo; back up real files |
| system | copy units, hooks, udev rules, sysctl, zram, helper scripts, sudoers drop-ins (validated with `visudo -c` first) | `cmp` with the installed copy; `daemon-reload` only if a unit changed |
| services | enable units of `config/services.txt` | `systemctl is-enabled` |
| snapper | create the snapshot configuration | `snapper get-config` |

**Table 2:** Stages of `install.sh`.

## 1.3 Desktop: i3, justified with data

i3 was compared with XFCE 4.20 installed temporarily. Both sessions were started identically (fresh boot, tty1 login, `startx`; the session is chosen by `~/.config/neonarch/session`) with only one xterm open; `scripts/measure-desktop.sh` waits 60 s and takes five samples 10 s apart.




| | i3 | XFCE |
|---|---|---|
| Used memory (MiB) | 598 | 845 |
| User processes | 25 | 41 |
| User RSS (MiB) | 245 | 776 |
| User PSS (MiB) | 143 | 356 |

**Figure 1:** Idle desktop footprint (mean of 5 samples; `docs/evidence/desktop/`).

XFCE adds 16 always-running processes (session manager, `xfdesktop`, `xfsettingsd`, `xfconfd`, panel and plugins, Thunar daemon, power manager, polkit agent, accessibility bus) and 2.5× the session's proportional memory. PSS is the fair metric because RSS counts shared GTK libraries once per process. Two measurement artefacts were identified and excluded in the discussion: the Alt+F2 launcher left running (42 MB) and a VirtualBox guest helper that grew to 112 MB only in the XFCE session. Even without them XFCE uses about 2.5× the RSS of i3. A lower baseline matters because Topic 2 creates memory pressure on a 4 GiB VM.

**Terminal.** Alacritty (GPU-rendered) hung without drawing a window: VirtualBox's VMSVGA adapter has no 3D support. Every *Super+Enter* left a stuck process (21 of them; RAM grew from 178 MiB to 1.6 GiB). From tty2 we proved the key arrived (`showkey`: keycode 125) and replaced the terminal by xterm, rendered on the CPU.

# 2 Layer 2 — Operating-system topics

Seven of the nine topics were implemented (Table 3). Topics 5 (IPC) and 8 (I/O and devices) were left out: inside a VM the GPU/NPU and hot-plug evidence of Topic 8 would be limited to virtual devices. Each topic has an evidence sheet `docs/tema-XX-*.md` with the five elements of Annex 2 and the raw data in `docs/evidence/`.

| Topic | Owner | Main artefacts | Headline result |
|---|---|---|---|
| 1 Processes | Hernán | `neon-ps`, `tema01-experiment.py` | shares 69.8/22.8/7.4 % vs 69.7/22.8/7.5 % |
| 2 Memory | Royner | zram + swapfile, stress-ng/vmstat | with zram: `so` −64 %, work −13 % |
| 3 Filesystem | Hernán | snapper, snap-pac, fio, rollback demo | writes 3.2–3.8× slower on CoW |
| 4 Services | Hernán | `neon-health.service` + timer | quota 20 % → 19.8 % measured |
| 6 Packages | Hernán | ALPM hook, `neonarch-tools` PKGBUILD | hook in every transaction |
| 7 Boot | Hernán | boot measurement, 2 optimisations, rescue | lz4 +10 % slower; defaults kept |
| 9 Security | Royner | nftables, SSH, sudoers, ACL | default deny; keys only; 2 tests to redo |

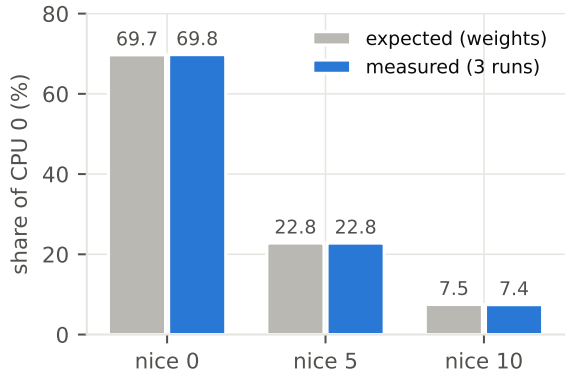
**Table 3:** Implemented topics.

## 2.1 Topic 1 — Processes and scheduling

**Implementation.** `neon-ps` lists PID, PPID, state, nice, policy, real-time priority, last CPU and %CPU (two samples of `utime+stime` from `/proc/<pid>/stat`) and changes nice (`setpriority`), policy (`sched_setscheduler`) and affinity (`sched_setaffinity`). `neon-boost` raises the nice value and I/O priority of every thread of a process (`/proc/<pid>/task`, `ioprio_set`).

**Concept.** Linux 7.2 uses EEVDF for the fair class: each task's virtual runtime advances inversely to its weight (nice 0 = 1024, each step ≈ ×1.25) and, among eligible tasks, the one with the earliest virtual deadline runs. Real-time classes always preempt the fair class.

**Method.** Three CPU-bound loops pinned to CPU 0 with nice 0, 5, 10; then a `SCHED_FIFO` loop (priority 10) against a normal one; 3 runs of 20 s, CPU time read from `/proc`.




| SCHED_FIFO vs SCHED_OTHER | FIFO | OTHER |
|---|---|---|
| run 1 | 94.10 % | 4.98 % |
| run 2 | 94.00 % | 5.05 % |
| run 3 | 93.99 % | 5.04 % |

**Figure 2:** Left: share of one core by nice value. Right: real-time vs fair class on one core.

**Interpretation.** The fair shares match $w_i / \sum w$ to $\pm 0.1$ percentage points: nice does not give absolute priority but a proportional share, and nice 10 is never starved. In the second experiment the FIFO task dominates, yet the normal task received exactly 5 %. On this kernel `sched_rt_runtime_us` equals the period (classic RT throttling disabled); the 5 % comes from the *fair server*, a deadline server that reserves 50 ms per second for fair tasks on each CPU. We verified it in `/sys/kernel/debug/sched/fair_server/cpu0`: runtime 50 000 000 ns, period 1 000 000 000 ns. Our script's "expected ≤95 %" column assumed the old mechanism, which the measurement corrected.

## 2.2 Topic 2 — Memory

**Implementation** (on `neonarch-roy`, 2 GiB RAM). zram via `zram-generator` (RAM/2 ≈ 970 MiB, zstd, priority 100) and a 2 GiB NOCOW swapfile on `@swap` (default priority): the kernel uses the highest-priority swap first, so pages go to compressed RAM (microseconds of CPU) before the disk (milliseconds of I/O). Tuning for zram: `vm.swappiness=180`, `vm.page-cluster=0`.

**Method.** `scripts/measure-swap-zram.sh`: `stress-ng --vm 1 --vm-bytes 3900M --vm-keep` for 180 s (capped at 1.76 GB with 1.61 GB available, so the run must page), once without zram and once with it; `vmstat -w 2` (100 samples per run), `free`, `swapon --show`.

| Metric (mean over the run) | Without zram | With zram | Change |
|---|---|---|---|
| `vmstat si` (KiB/s swapped in) | 15.5 | 23.6 | +52 % |
| `vmstat so` (KiB/s swapped out) | 226.0 | 80.6 | −64 % |
| `wa` (% I/O wait) | 0.0 | 0.0 | — |
| stress-ng throughput (bogo ops/s) | 1908.7 | 1654.8 | −13 % |

**Table 4:** Memory pressure with and without zram (`docs/evidence/tema2-neonarch-roy/`).

**Interpretation.** zram changes the paging profile rather than speeding the load up: with zram the swap-out rate fell by 64 % and swap-in rose, because evicting a page to compressed RAM is cheap. vmstat's `si`/`so` count traffic to *any* swap device, zram included, so they do not isolate disk traffic; the swapfile still ended with 68 MiB used. The cost is CPU: 13 % less work, consistent with zstd on 2 vCPUs. `wa` stayed at 0 because the virtual disk sits on the host's NVMe SSD, which hides the I/O penalty zram avoids on slow disks. No OOM kill occurred. zram pays off when the disk is slow or must not wear; with a fast disk and a busy CPU its benefit is smaller.

## 2.3 Topic 3 — Filesystem (btrfs)

**Implementation.** btrfs root with the subvolume layout of Table 1; snapper with hourly timeline snapshots (5 hourly + 7 daily kept) and `snap-pac`, which takes a *pre* and a *post* snapshot around every pacman transaction; shortcut *Super+Shift+S* for a manual snapshot.

**Method.** fio with `O_DIRECT`, `io_uring`, depth 16, 1 GiB, 3 runs of 20 s per workload. The ext4 reference is a loop image *on the same btrfs root* created NOCOW (`chattr +C`) and attached with `losetup --direct-io=on`.

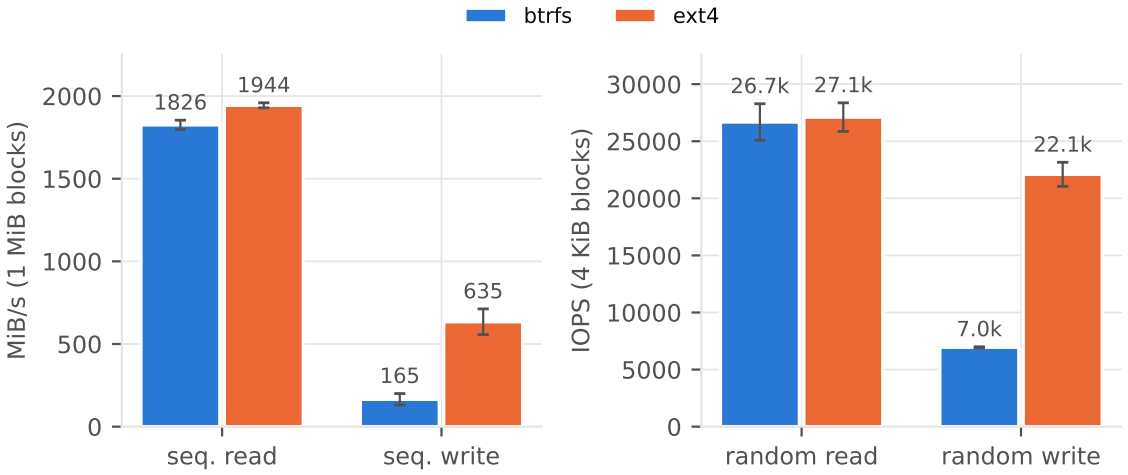



**Figure 3:** btrfs (CoW, zstd:1) vs ext4 on the same virtual disk (mean ± sd of 3 runs).

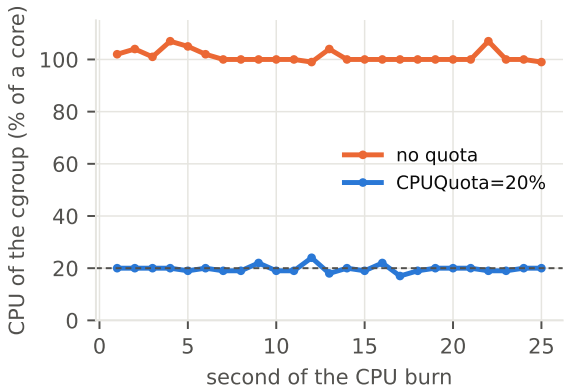
**Interpretation.** Reads are equivalent (within 2–6 %): reading changes nothing, so copy-on-write costs nothing. Writes are 3.8× (sequential) and 3.2× (random, 2.29 vs 0.72 ms per I/O) slower on btrfs: every write allocates a new extent, stores a checksum and updates the B-trees up to the root, and metadata is duplicated (`DUP`). The NOCOW image sits on the same filesystem and device and writes 3–4× faster, which isolates CoW as the cause. We still chose btrfs because a desktop is read-dominated and CoW gives snapshots, rollback and checksums; write-heavy files (swap, VM images) belong in NOCOW subvolumes, as we did with `@swap`.

**Discarded measurement.** The first run gave ext4 5 785 MiB/s and 217 914 random-read IOPS (71 µs): impossible for this disk. The loop device was buffered, so ext4 was reading the host's page cache. The data were kept (`fio/discarded-buffered-loop/`) and the benchmark was repeated with a direct-I/O loop.

**Rollback demo.** Removing `htop` with pacman created snapshots #22 (pre) and #23 (post). `snapper status 22..23` showed the deleted binary, man page, icons *and pacman's database entry*; `snapper undochange 22..23` restored them (`create:9 modify:2`) and `pacman -Q htop` recognised the package again. `pacman -Qkk` then reported 5 files with only a modification-time mismatch: `undochange` is a file-level copy. An exact whole-system rollback is `snapper rollback` (switching the default subvolume).

## 2.4 Topic 4 — Service management (systemd)

**Implementation.** `neon-health.service`: a daemon that samples `/proc` every 10 s into its state directory, with `After=local-fs.target`, `Wants=` a report timer (hourly, `Persistent=true`), `Restart=on-failure` with a start-rate limit, `CPUQuota=20%`, `MemoryMax=64M`, `MemorySwapMax=0`, and sandboxing (`DynamicUser`, `ProtectSystem=strict`, `ProtectHome`, `NoNewPrivileges`). Signals `SIGUSR1`/`SIGUSR2` trigger a CPU burn and an unbounded allocation to test the limits.




| Test | Result |
|---|---|
| CPU burn, `CPUQuota=20%` | 19.8 % |
| CPU burn, quota removed | 101.2 % |
| allocate beyond 64 MiB | OOM kill, restart (0→1) |
| `SIGKILL` | restart (1→2) |

**Figure 4:** CPU of the service's cgroup (`cpu.stat` sampled every second) and failure tests.

**Interpretation.** systemd places each service in its own cgroup v2. `CPUQuota=20%` becomes `cpu.max =`

`20000 100000`: the group may run 20 ms per 100 ms period and is then throttled. This is a hard cap, unlike nice, which only shares the CPU when there is contention. `MemoryMax` is enforced by the memory controller: the OOM killer acted *inside* the cgroup only, and `Restart=on-failure` brought the service back. The service never runs as root.

## 2.5 Topic 6 — Package management (pacman)

**Implementation.** An own ALPM hook (`PostTransaction`, `NeedsTargets`) logs every transaction, lists orphan packages and warns when the running kernel or systemd were upgraded. The PKGBUILD `neonarch-tools` packages our five commands with `makepkg`.

**Results.** pacman 7.1.0 with `SigLevel = Required DatabaseOptional` and 183 trusted keys; the local database holds 416 packages, one directory each (`desc`, `files`, `mtree`). Our package: version 1.0.0-2, 11 files, `pacman -Qo /usr/bin/neon-info` reports ownership and `pacman -Qkk` reports 0 altered files. In every transaction the order was: snap-pac pre-snapshot → package changes → our hook → snap-pac post-snapshot. `pacman -Sp xfce4-terminal` resolved 9 dependencies without installing anything.

**Interpretation and a lesson.** Packaging gives what copying files cannot: ownership (`-Qo`), integrity checks against the MTREE (`-Qkk`), upgrades and clean removal. The hook order makes every transaction reversible (Topic 3). The first build packaged four *empty* commands: terminal output had been pasted back into bash, and lines containing > truncated the files. `pacman -Ql` could not reveal it, but `tar -tvf` of the package showed 0 bytes. We restored the files from Git, rebuilt (`pkgrel=2`) and disabled xterm's copy-on-select.

## 2.6 Topic 7 — Boot process

**Boot chain.** UEFI firmware (NVRAM entry *Linux Boot Manager*) → `systemd-bootx64.efi` reads `loader.conf` and `neonarch.conf` → loads `vmlinuz-linux` and `initramfs-linux.img` from the ESP with `root=UUID=... rootflags=subvol=@ rw` → the kernel unpacks the initramfs (systemd-based, mkinitcpio hooks) → udev loads the disk and btrfs drivers, the root is mounted and `switch_root` runs → systemd (PID 1) reaches `graphical.target` → getty → login → `startx` → i3.

**Method.** `systemd-analyze time`, `blame`, `critical-chain` and `plot` after each boot, 3 boots per variant. Two optimisations of the initramfs were tried: lz4 compression (faster decompression) without the useless `fsck` hook (btrfs), and a "slim" image (zstd, without `fsck` and `kms`).

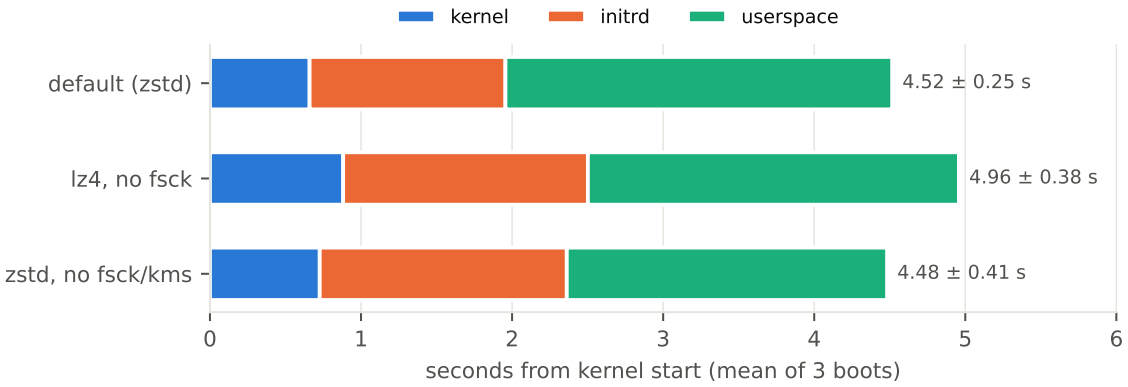



**Figure 5:** Boot phases per initramfs variant (mean of 3 boots; total mean ± sd). Image sizes: 15.2, 21.9 and 14.9 MB.

**Interpretation.** lz4 made the boot *slower* (+0.55 s in kernel+initrd, very consistent): its image is 44 % larger, and on a Ryzen CPU the extra I/O costs more than the faster decompression saves. The slim image (only 2 % smaller) was within the noise of the default. The critical chain is dominated by device discovery (`.device` units ≈1.9 s in `blame`), not by the initramfs. We kept the Arch defaults: two plausible optimisations were *rejected with data*. The NVRAM boot entry registered at install time was later lost by VirtualBox (`efibootmgr`); the VM keeps booting through the fallback copy `\EFI\BOOT\BOOTX64.EFI`. The firmware and loader phases are not reported in this VM (no `LoaderTime*` EFI variables), so the menu timeout change could not be measured. The first recorded boot (just after a system upgrade, with `ldconfig` in the critical chain) was discarded.

**Rescue demo.** Extra boot entries add `systemd.unit=rescue.target` and `emergency.target`; `bootctl set-oneshot` selects one for the next boot only. In rescue, `sulogin` asks for the root password

and only 11 targets are active (sysinit, local-fs, swap, rescue…): filesystems mounted, but no `basic`, `network`, `multi-user` or `graphical` target. `systemctl default` continues the normal boot.

### 2.7 Topic 9 — Protection and security

**Implementation** (on `neonarch-roy`). nftables with *input policy drop* (allowed: loopback, established/related, ICMP, TCP 22; invalid dropped); sudoers where `roy` may run *without password* only `systemctl restart/status sshd` and pacman queries (`-Q`, `-Qi`, `-Ql`), all files validated with `visudo -c`; ACLs on `/srv/neonarch-shared` (owner rwx, user `tester` rwx, nobody else, with default ACLs inherited by new files); SSH with `PermitRootLogin no`, `PasswordAuthentication no` (ed25519 key), `MaxAuthTries 3`, `X11Forwarding no`. Plus the sandbox of Topic 4 and the root-only ESP.

**Results.** `nft list ruleset`, `sshd -T`, `getfacl` and `visudo -c` confirm the configuration in force; a file created by `tester` inherited the ACL (`-rw-rw---+`). Two tests are not valid yet and are reported as such: the ACL "cannot delete" test targeted a file that did not exist, and the recorded `nmap` scan was run against Nmap's public test host (`scanme.nmap.org`) instead of the VM. Both must be repeated against `neonarch-roy`.

**Interpretation.** The layers are independent (defence in depth): with `policy drop` an unlisted port gives no answer, so nmap reports it as *filtered* rather than *closed*; without passwords, SSH brute force has nothing to guess; sudo limits what runs without authentication. `roy` remains in `wheel` (`ALL` with password, kept for the project's scripts), so the restriction covers password-less execution, not everything; a stricter design removes the user from `wheel`.

## 3 Layer 3 — Own commands and shortcuts

| Command | Author | OS interfaces |
|---|---|---|
| `neon-info` (mandatory) | Hernán | `/proc/{meminfo, swaps, cpuinfo, loadavg, uptime, [pid]/stat, self/mounts}`, `/sys/{bus/pci, block/zram*, class/accel, devices/system/cpu, firmware/efi}`, `uname`, `statvfs`, `sched_getscheduler`; GPU/NPU with driver from sysfs (`vmwgfx` here); `-sources` prints the interface behind each value |
| `neon-ps` | Hernán | `/proc/[pid]/stat`, `setpriority`, `sched_setscheduler`, `sched_setaffinity` |
| `neon-boost` | Hernán | `/proc/[pid]/task`, `setpriority`, `ioprio_set`; restricted sudoers rule |
| `neon-net` | Hernán | `/proc/net/dev`, `/sys/class/net`, `ioctl(SIOCGIFADDR)` |
| `neon-mem` | Hernán | `/proc/{meminfo, swaps, vmstat, [pid]/stat}`, `/sys/block/zram*/mm_stat` (Topic 2 monitor) |
| `neonarch-health` | Royner | health check: `/proc`, `/sys`, `systemctl -failed`, `journalctl -p err`, `btrfs filesystem usage`, snapshots, orphans; exit code 0/1/2 |
| `neonarch-clean` | Royner | safe cleanup, dry-run by default: pacman cache, journal vacuum, old snapshots, orphans; btrfs usage before/after |
| `neonarch-calc` | Royner | terminal calculator |

**Table 5:** Own commands (Hernán's are also packaged as `neonarch-tools`, Topic 6).




**Figure 6:** `neon-info` on `neonarch-hernan`: every value is read from `/proc`, `/sys` or a system call (`neon-info -sources` prints which). It reports the EEVDF scheduler, the virtual GPU with its driver (`vmwgfx`), zram and the btrfs root subvolume.

Mandatory shortcuts: *Super+Enter* terminal, *Super+W/Q* close, *Super+D* rofi, *Super+1…9* workspaces. Hernán's: *Super+I* `neon-info` in a floating window, *Super+B* `neon-boost` on the focused window, *Super+Shift+S* manual btrfs snapshot with a notification. The last one collided with the host: Windows' screenshot shortcut (Win+Shift+S) reached the VM first and created snapshots #17 and #19 while we were taking screenshots. Royner's: *Super+Shift+L* lock screen (`i3lock`), *Super+Shift+K* `neonarch-calc`, *Super+Shift+P* area screenshot (`maim -s`). Full list: `docs/cheatsheet.md`.

# 4 Limitations

- **Virtualisation.** All measurements run in VirtualBox over a VDI file. The host's cache and SSD, and the host's own activity, add noise. Boot times varied by up to 0.4 s between identical boots. Firmware/loader times and real GPU/NPU behaviour are not observable.
- **Repetitions.** 3 runs per benchmark and 3 boots per variant are enough to see large effects (CoW writes, lz4) but not small ones (the "slim" initramfs).
- **Synthetic loads.** fio writes incompressible random data, which removes the benefit of `compress=zstd`; stress-ng memory pages compress far better than real data.
- **Open items.** Topic 9's external nmap scan and ACL denial test must be repeated against the VM (see Topic 9).
- **Scope.** Topics 5 and 8 were not implemented; the snapper rollback shown is file-level (`undochange`), not a full subvolume rollback.

# 5 Lessons learned

- **Measure, then believe.** Two optimisations that sound right (lz4, slim initramfs) were not, and the scheduler did not follow the textbook RT-throttling model on a 7.x kernel. The data showed it.
- **Sanity-check every number against the hardware.** 5.7 GiB/s from a virtual disk meant we were measuring RAM.
- **Verify the artefact, not only its metadata.** The empty package passed `pacman -Qi`; `tar -tvf` caught it.
- **Know the tool's layer.** bash's command hash made a deleted program look present; xterm's copy-on-select plus pasting output into a shell destroyed files; the host and the guest shared a keyboard shortcut.

- **Use of AI.** We used Claude as a tutor and generator of first drafts (`docs/bitacora_ia.md`, 8 entries). Five AI errors were detected by measuring or reading the documentation: proposing an ISO that the assignment does not ask for, trailing comments that broke the i3 config, a GPU terminal unusable without 3D, a buffered loop device that biased the benchmark, and the bash-hash problem in the rollback demo. Each one was a lesson about the OS, not only about the tool.

# 6 Conclusions

NeonArch is reproducible (manual installation documented step by step, idempotent bootstrap) and every topic is backed by measurements that can be repeated with one script. The measurements confirmed the theory where it applies (proportional fair shares, cgroup limits, the cost of copy-on-write), showed where the theory has moved on (fair server instead of RT throttling), and rejected changes that would only have been assumed to help.

# References


- A. Silberschatz, P. B. Galvin, G. Gagne, *Operating System Concepts*, 10th ed., Wiley, 2018: ch. 2, 3, 5, 9–11, 13–17.
- Arch Wiki: Installation guide, Btrfs, Snapper, systemd-boot, Mkinitcpio, zram, pacman/Tips, nftables, OpenSSH. `https://wiki.archlinux.org`
- Manual pages: `sched(7)`, `systemd.resource-control(5)`, `systemd.service(5)`, `bootup(7)`, `alpm-hooks(5)`, `PKGBUILD(5)`, `fio(1)`, `snapper(8)`, `proc(5)`.
- Linux kernel documentation: *EEVDF Scheduler*; *Deadline Task Scheduling* (fair server); *Control Group v2*. `https://docs.kernel.org`